

eTimeTrackLite

Device Monitor — Automated Screenshot & Email

Project Report

Organisation	RentoMojo
Team	IT Department
Prepared By	Charan Bijapur
Date	March 2026
Version	1.0 — Final
Status	 Working & Deployed

1. Executive Summary

This project delivers an automated biometric device monitoring solution for RentoMojo. The system logs into the eSSL eTimeTrackLite Server, captures a full-page screenshot of the Device Status dashboard, and emails it to the IT team every 3 hours during business hours (09:00–20:00 IST). This eliminates the need for manual checks and ensures the team is immediately informed of any offline biometric devices across all locations.

2. Problem Statement

Before this solution, the IT team faced the following challenges:

- No automated way to monitor 16+ biometric attendance devices across India
- Device outages were detected only after employee complaints — causing attendance data loss
- Manual login to the eTimeTrackLite portal was required to check device status
- No scheduled reporting mechanism existed for device health
- Offline devices went unnoticed for hours, affecting payroll accuracy

3. Solution Overview

The solution is a Python automation script that uses Selenium WebDriver to programmatically control a Chrome browser, log into the eTimeTrackLite portal, capture the dashboard screenshot, and deliver it via email. The script runs continuously on a Windows server with a built-in scheduler.

Step-by-step flow:

- Script launches Chrome browser (off-screen, not visible to user)
- Navigates to eTimeTrackLite login page automatically
- Enters credentials and submits the login form
- Waits for dashboard to fully load (10 seconds)
- Takes full-page screenshot of the Device Status dashboard
- Sends email with screenshot embedded to IT team
- Repeats every 3 hours between 09:00 and 20:00 IST

4. Technical Details

4.1 Technology Stack

Component	Technology	Purpose
-----------	------------	---------

Language	Python 3.x	Core scripting language
Browser Control	Selenium WebDriver	Automates Chrome browser
Browser	Google Chrome	Renders the eSSL web portal
Driver Manager	webdriver-manager	Auto-downloads ChromeDriver
Scheduler	schedule library	Runs every 3 hours automatically
Email	smtplib + Gmail SMTP	Sends screenshots via email
Screenshot	Selenium save_screenshot	Captures full dashboard image
Anti-detection	CDP + Custom User-Agent	Bypasses headless browser block

4.2 Key Technical Challenges & Solutions

Challenge	Solution
Server blocks headless Chrome	Removed --headless flag entirely. Chrome runs off-screen using --window-position=-32000,-32000 so it is invisible to the user but not detectable as headless by the server.
Session lost when navigating to device status page	The eSSL portal uses a frameset (Main.aspx). Direct navigation to devicestatus.aspx destroyed the session. Fixed by taking screenshot immediately after login on Main.aspx — which already shows the full dashboard.
Login form selectors not matching	Used debug script to dump all input field names. Confirmed exact selectors: StaffloginDialog\$txt_LoginName, StaffloginDialog\$txt_Password, StaffloginDialog\$Btn_Ok.
Screenshot capturing wrong page	Simplified the flow — login, wait 10 seconds, screenshot. No extra navigation needed.
Anti-automation detection	Added custom User-Agent string, removed automation flags via CDP command, set navigator.webdriver = undefined via JavaScript injection.

5. Setup & Installation Guide

5.1 Prerequisites

- Windows 10/11 PC or Server (must stay ON during monitoring hours)
- Python 3.8 or higher installed
- Google Chrome browser installed
- Internet access to reach <http://35.154.100.94:99>
- Gmail account with App Password enabled (2FA required)

5.2 Installation Steps

Step 1 — Install Python dependencies:

```
pip install selenium webdriver-manager schedule
```

Step 2 — Save the script:

Save etime_monitor.py to: C:\Users\Admin\Desktop\Project-ESSL\

Step 3 — Run the script:

```
cd C:\Users\Admin\Desktop\Project-ESSL
python etime_monitor.py
```

Step 4 — Auto-start on Windows boot (optional):

- Press Win + R, type shell:startup, press Enter
- Create a shortcut to etime_monitor.py in that folder
- The script will start automatically when Windows boots

5.3 Configuration

All settings are in the CONFIG section at the top of etime_monitor.py:

Setting	Current Value	Description
app_url	http://35.154.100.94:99/...	eTimeTrackLite login page URL
username	ESSL	Login username
password	***** (encrypted)	Login password
sender	charan.bijapur@rentomojo.com	Gmail sender address

app_password	16-char App Password	Gmail App Password (not Gmail password)
recipients	it@rentomojo.com + 1 more	Email recipients list
schedule	Every 3 hours	09:00, 12:00, 15:00, 18:00, 20:00

6. Output & Results

6.1 Email Report

- Subject: eTimeTrackLite - Device Status Report DD-Mon-YYYY HH:MM
- Recipients: it@rentomojo.com, yogesh.patel@rentomojo.com
- Content: Full dashboard screenshot embedded in HTML email
- Dashboard shows: Registered Employees, Active Employees, Present Employees, Online Devices, Offline Devices
- Device table: DeviceSName, DeviceFName, Serial No., Location, Last Ping, Status (online/offline)

6.2 Devices Monitored

The dashboard monitors biometric attendance devices across all RentoMojo locations:

Location	Location	Location
Pune (Incitey)	Bangalore WH	Bangalore HO
Ahmedabad	Gurugram	Delhi WH
Indore	Taloja	Kolkata Incitey
Kolkata New WH	Hosakote WH	Chennai
Noida	Jaipur	Hyderabad
Lucknow	—	—

6.3 Log Files

All runs are logged to: logs/etime_monitor.log

- Timestamp of every run
- Login success/failure status
- Screenshot file path and size
- Email delivery confirmation
- Any errors with full stack trace

7. Business Benefits

Before (Manual)	After (Automated)
✗ Manual login required to check status	✓ Fully automated — zero manual effort
✗ Outages detected hours after occurrence	✓ Team notified within 3 hours maximum
✗ No historical records of device health	✓ Complete log file maintained automatically
✗ Dependent on staff availability	✓ Runs 24x7 without human intervention
✗ No centralised visibility	✓ Email report to all stakeholders simultaneously

8. Future Improvements

- WhatsApp / Slack alert when a device goes offline
- Automated ticket creation in JIRA when device offline > 1 hour
- Dashboard with historical uptime graphs per location
- SMS alert to location manager for their specific offline device
- Weekly uptime report with percentage per device
- Auto-restart script if Chrome crashes

9. Conclusion

The eTimeTrackLite Device Monitor project successfully automates biometric device health monitoring for RentoMojo's 16+ locations across India. The solution required overcoming significant technical challenges including headless browser detection, session management in frameset-based web applications, and login form automation. The final working solution is simple, reliable, and maintainable — it runs continuously with no manual intervention and delivers actionable device status reports to the IT team every 3 hours.

Project Status:  **Successfully deployed and working.**